

Cinamate – Plan Complet

Tinder pentru filme/seriale. Doi useri fac un profil de gusturi, se conectează printr-un cod de invite, primesc împreună un deck de sugestii, fiecare face swipe, iar când amândoi dau swipe-dreapta pe același titlu → **match** = filmul câștigător. Merge și solo.

1. Ce vrem să facem

- Quiz / import de date → profil de gusturi per user
- Cod de invite → cameră privată între doi useri (sau grup, v3)
- Swipe deck: poster mare, titlu, gen, descriere scurtă
- Filtrare pe platformă de streaming (Netflix, HBO, Disney+) sau "orice/Plex"
- Toggle film ⇌ serial

- Mod solo (când nu știi ce să vezi și ești singur)
 - Match instant, live (fără refresh), cu ecran dedicat "filmul câștigător"
-

2. Unde hostăm — 100% free tier Cloudflare

Piesă	Serviciu	Rol
Frontend	Cloudflare Pages (static)	UI-ul de swipe
API	Cloudflare Workers (Hono)	Rutare, logică, auth simplu
DB relațională	D1 (SQLite)	users, profiles, rooms, swipes, matches
Stare live per cameră	Durable Objects	sesiunea de swipe a doi useri, WebSocket, calcul match instant

Piesă	Serviciu	Rol
Cache	Workers KV	răspunsuri TMDb (postere, detalii) – evită rate-limit
Date filme	TMDb API	postere, titlu, gen, descriere, watch providers (Netflix/HBO/Disney/etc)

Costuri: zero, pentru scenariul tău (tu + o persoană, eventual câțiva prieteni). Limitele free tier (Workers 100K req/zi, D1 5GB/5M citiri/zi, Durable Objects ~100K req/zi, KV 1GB/100K citiri/zi) sunt cu mult peste orice ai nevoie. TMDb API e gratuit pentru uz non-comercial. Ai plăti doar dacă explodează traficul la mii de useri activi zilnic, sau dacă vrei Cron Triggers (dar și acelea au variante gratuite dacă faci refresh "lazy", la cerere).

De ce nu scraping pe IMDb/Letterboxd: ambele interzic explicit scraping

în ToS și au protecție anti-bot serioasă (IMDb mai ales). TMDb API acoperă postere, titlu, gen, descriere, watch providers per regiune (RO) – exact ce ne trebuie, gratuit, fără riscul de a fi blocați.

3. Structura proiectului

```
cinemate/
├── PLAN.md
├── wrangler.toml          # config
Workers + D1 + D0 + KV bindings
├── package.json
├── schema.sql             # schema
D1
├── src/
│   ├── index.ts           #
Worker principal, rute Hono
│   ├── types.ts           #
tipuri comune (Env, Profile, Room,
DeckCard...)
│   ├── services/
│   │   └── tmdb.ts         #
integrare TMDb + cache KV
│   └── durable-objects/
│       └── room.ts         #
stare live camera (swipe, match,
WebSocket)
```

```
└─ public/                                     #
frontend static (Pages)
├─ index.html
├─ style.css
└─ app.js
```

Schema DB (D1)

- **users** — id, username, created_at
- **profiles** — user_id, genre_scores (JSON), era_pref, mood_pref, media_type_pref
- **rooms** — id, join_code, user_a_id, user_b_id (null = solo), platform_filter, media_type, status
- **swipes** — room_id, user_id, tmdb_id, media_type, direction (like/dislike)
- **matches** — room_id, tmdb_id, media_type, matched_at

Rute API (Worker)

- `POST /api/users` — creează user
- `POST /api/profile/quiz` — salvează profilul de gusturi
- `POST /api/rooms` — creează cameră + cod de invite

- `POST /api/rooms/join` — intri într-o cameră cu cod
 - `GET /api/rooms/:id/deck` — deck de titluri, filtrat pe profil combinat + platformă
 - `POST /api/rooms/:id/swipe` — înregistrează swipe, calculează match live prin Durable Object
 - `GET /api/rooms/:id/matches` — lista de match-uri + unde poate fi văzut fiecare
 - `GET /api/rooms/:id/ws` — conexiune WebSocket pentru notificări live de match
-

4. Flow-ul aplicației

1. **Onboarding:** quiz (genuri, mood, epocă, comedie vs dramă) SAU import CSV din Letterboxd (vezi secțiunea 6) → generează `genre_scores`.
2. **Creare cameră:** user devine `user_a`, alege platformă + `media_type`, primește `join_code` (6 caractere).
3. **Join:** al doilea user intră cu codul → devine `user_b` → camera `active`.

4. **Deck:** Worker combină `genre_scores` ale ambilor (medie ponderată), cere `discover` din TMDb filtrat pe top genuri + watch-provider + media_type, exclude tot ce a fost deja swipe-uit în cameră.
 5. **Swipe:** fiecare swipe merge în D1 și către Durable Object-ul camerei; DO ține swipe-urile în memorie și, dacă ambii au dat like pe același titlu, creează match și trimite live prin WebSocket la ambii clienți (fără refresh).
 6. **Match screen:** poster mare + unde poate fi văzut (watch providers).
 7. **Solo mode:** cameră fără `user_b` — orice like = salvat direct ca "match"/watchlist personal.
 8. **Toggle film/serial:** schimbă `media_type` pe cameră → deck-ul se regenerează pe `/movie` sau `/tv`.
-

5. Numele — Cinemate

Cinema + mate (prieten/partener). Sună bine și în română și în engleză, descrie exact funcția (prietenul tău de cinema), scurt și ușor de reținut.

6. Ce aduce Cinemate în plus față de aplicațiile existente

Există deja aplicații similare pe piață (Netflip, MatchaFilm, Movie Swiper, Matched, Reelgood's Swipe With Friends) — toate sunt aplicații închise, black-box, gândite pentru userul general. Cinemate, fiind self-hosted și construit pe infra proprie (ai deja homelab + arr stack), poate ataca un unghi pe care ele nu-l ating: **userul tehnic, self-hoster**.

Diferențiatori posibili

1. **Integrare cu Plex** — verifică dacă filmul e deja pe serverul propriu (via Plex API) și marchează "ai deja asta".

2. **Transparență pe algoritm** — arată explicit de ce a ieșit match-ul
("acțiune 80% + comedie 60%, ambii like la filme cu rating >7.5"), spre deosebire de recomandările black-box din aplicațiile comerciale.
3. **Auto-adăugare în Radarr/Sonarr la match** — dacă filmul nu există pe Plex, buton "adaugă la coadă" → trimite direct în Radarr/Sonarr.
Închide bucla completă: de la "ce vizionăm" la "gata, se descarcă".
4. **Fără cont, fără colectare centralizată de date** — self-hosted pe Cloudflare-ul propriu, poziționat explicit "datele tale rămân la tine".
5. **Mod grup, nu doar pereche** — extensie naturală: Durable Object-ul deja gestionează stare per cameră, se poate extinde de la 2 la N useri (match = majoritate sau unanimitate).
6. **Import din Letterboxd (CSV), nu doar quiz** — vezi mai jos.

Cel mai puternic combo ca diferențiator: **Plex-check + auto-queue** în

Radarr/Sonarr — unghiul "pentru oameni ca mine", pe care nicio aplicație din App Store nu-l atinge, pentru că nu au acces la un server self-hosted.

7. Integrare Letterboxd

Letterboxd nu are API public deschis (acces doar pentru parteneri aprobați), iar scraping-ul le încalcă ToS-ul și au protecție Cloudflare în față. Soluția curată:

Import din exportul CSV al userului — Letterboxd permite export propriu al diary/watched/ratings din Settings. Userul își descarcă fișierul, îl uploadează în Cinemate la onboarding → parser CSV → mapare titluri pe TMDb ID (căutare titlu+an) → construiește `genre_scores` automat din rating-uri reale, nu din preferințe declarate la un quiz. Se potrivește perfect cu diferențiatorul "datele tale rămân la tine" — userul își aduce singur datele, nu le extragem noi.

Opțional, cosmetic: link "vezi pe Letterboxd" lângă fiecare film din ecranul de match, fără să extragem date de acolo.

(Există librării neoficiale de scraping pentru profiluri publice Letterboxd, dar sunt fragile — se strică la orice schimbare de layout — și încalcă tot ToS-ul. Nu recomandate pentru un proiect pe care vrei să-l ții stabil.)

8. Roadmap

V1 — Schelet (făcut)

- Schema D1, Worker cu rute de bază, TMDb service, Durable Object pentru cameră, UI minimal de swipe (HTML/JS simplu, fără framework greu)

V2

- Quiz real cu scoring propriu-zis (înlocuiește profilul default hardcodat)
- Import CSV Letterboxd ca alternativă la quiz

- Watch-provider filtering complet funcțional
- Toggle film/serial funcțional (schimbă `media_type` real în DB)
- WebSocket live testat end-to-end pentru match
- UI mai finisat (posibil migrare pe React/Svelte pe Pages)

V3

- Integrare Plex (verifică ce ai deja pe server)
- Auto-queue în Radarr/Sonarr la match
- Mod grup (3-5 useri, nu doar pereche)
- Collaborative filtering ușor, bazat pe istoricul de swipe-uri comune
- Watchlist persistent pentru solo mode
- Autentificare reală (momentan doar cod de invite + username)

9. Pași ca să pornești local

```
cd cinemate
npm install
npx wrangler d1 create cinemate-db
# pune id-ul in wrangler.toml
```

```
npx wrangler kv:namespace create  
TMDB_CACHE # pune id-ul in  
wrangler.toml  
npx wrangler d1 execute cinemate-db --  
file=schema.sql  
npx wrangler secret put TMDB_API_KEY  
# cheia de la themoviedb.org  
npx wrangler dev
```